

System Design Document

Team: LionsList

Members: Isha Chaudhry, Raymond Au, Gurleen Sandhu, Andrei Lopez, Justin Valle Amaya

York University

EECS3311: Software Design

Iir Dema

April 6, 2026

Table of Contents

1. Introduction.....	2
2. System Environment.....	3
3. System Architecture.....	4
4. System Decomposition.....	5
4.1 Frontend Components	
The frontend for LionsList is a React single-page application (SPA) that provides the user interface for several marketplace features:.....	5
4.2 Backend Components.....	5
4.3 Application Routes.....	8
4.4 Database Models.....	9
5. CRC Cards.....	10
6. Error Handling Strategy.....	13
6.1 Invalid User Input.....	13
6.2 Authentication and Authorization Errors.....	13
6.3 Database Errors.....	14
6.4 Network and API Communication Errors.....	14
6.5 Data Integrity Protection.....	14

1. Introduction

LionsList is a student-focused web-based marketplace designed exclusively for York University to buy and sell academic materials within the York University Community.

General marketplaces such as Kijiji and Facebook Marketplace do allow people to list textbooks and academic supplies but are not specifically aimed at university students. These platforms do not have the ability to filter by course, nor the verification of the users. Consequently, students frequently come across irrelevant listings, unreliable buyers, scams, and low value offers on these platforms.

The official York University bookstore is where students can get the academic materials they need. However, prices are usually fixed at high retail prices paired with a strict refund, return, and exchange policy. Online forums like the unofficial York University subreddit provide a medium for students to communicate, but lack structured listings, filter functionality, and user verification.

LionsList addresses these limitations by providing:

- York University Exclusive Access
- Course-code Filtering
- User Verification
- Searchable Academic Materials Listings
- Flexible Student Pricing

2. System Environment

Runtime Environment

- Node.js (LTS version)
- MongoDB database
- Web browser for accessing the client application

Programming Technologies

- Frontend: React, Axios HTTP client, JavaScript / HTML / CSS
- Backend: Node.js, Express.js framework, Mongoose ODM for MongoDB
- Database: MongoDB

Project Structure

The repository contains two main components:

client/

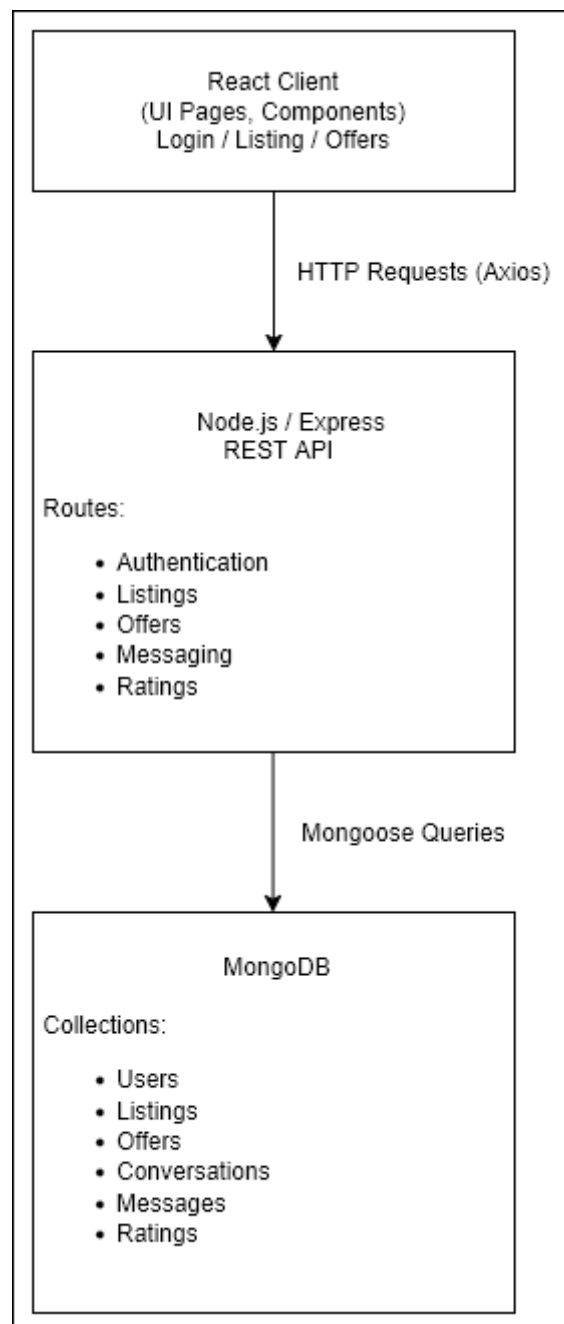
- React frontend application
- Sends HTTP requests to backend API

server/

- Express backend API
- Handles business logic
- Communicates with MongoDB

Environment configuration is handled through a .env file in the server directory.

3. System Architecture



The LionsList system follows a three-tier web architecture consisting of a presentation layer, an application layer, and a data layer.

- The React frontend provides the user interface and communicates with the backend through HTTP requests using Axios.
- The Express backend processes these requests, applies business logic, and interacts with the MongoDB database through Mongoose models.
- MongoDB stores persistent application data such as users, listings, offers, conversations, and ratings.

4. System Decomposition

4.1 Frontend Components

The frontend for LionsList is a React single-page application (SPA) that provides the user interface for several marketplace features:

- Home page
- Authentication pages (Login / Register)
- Browse Listings page with a search a bar and filters
- My Listings page
- Listing Details page
- Create Listing form
- Edit Listing form
- Sent Offers Page
- Sent Offer Details Page
- Navigation bar (Header)

These components communicate with the backend using Axios API requests.

Technologies

- React
- Axios
- JavaScript
- HTML
- CSS

UI Composition

Header

- Header.jsx is a reusable navigation component displayed at the top of every page.
- Left side: LionsList branding
- Right side: Links for browsing listings, creating a listing, accessing sent offers, viewing one's own listings, signing in, signing up, and logging out (if logged in)

4.2 Backend Components

Authentication Flow

Register (Sign Up)

Data Collected:

- Full name (username)
- Email (@my.yorku.ca or @yorku.ca)
- Password

Backend Validation:

- Schema rules: uniqueness, length, format
- Password hashing
- JWT generated on successful signup

Frontend Validation:

- Full name ≥ 3 characters
- Email ends with @my.yorku.ca or @yorku.ca
- Password ≥ 8 characters
- Password and Confirm Password match

Sign In

Data Collected:

- Email (@my.yorku.ca or @yorku.ca)
- Password

Backend Validation:

- Checks email exists in DB
- Compares input password with hashed password
- JWT generated on successful login

Frontend Validation:

- Email exists
- Password matches stored password

Listing Features

Browse Listings

- Users can view all available listings
- Displayed on each listing card:
 - Image
 - Title
 - Description
 - Category
 - Course code (if provided)
 - Price
 - Status

Listing Details

- Detailed listing page shows:
 - Title
 - Description
 - Category
 - Course code (if provided)
 - Price
 - Status
 - Seller
 - Created timestamp

- Offers received (only for sellers)
- Edit function (only for sellers)
- Delete function (only for sellers)
- Make offer button (for non-sellers who are logged in)

Create a Listing

Data Collected:

- Title
- Description
- Category
- Course code (optional)
- Image URL (optional)
- Price
- Status (default: “Available”)
- Seller

Automatic Fields:

- createdAt and updatedAt timestamps
- Seller which is set from the logged-in user
- Status which defaults to “Available”

Edit a listing:

All fields from the Create Listing form are editable, along with the status

My Listings

- Shows all listings created by the logged-in user
- Each listing card shows the listing information and a “View Details” button, behaving similar to cards on the Browse page

Offer Features

Make Offer

- Available on Listing Details page for non-sellers who are logged in
- Offer amount must be a positive number with up to 2 decimal places
- Offer is saved with a “Pending” status by default

Sent Offers

- Shows all offers sent by the logged-in buyer
- Each listing card is displayed and behaves similar to the cards on the “Browse” and “My Listings” pages, but with an offer status badge (top right corner) as well as the offer amount

Sent Offer Details

- Displays full listings information for the offer, such as in the Listing Details Page

- Additionally, shows information for “Your Offer”, including the offer amount, the status, and a cancel offer button.

4.3 Application Routes

Route / Page	Description
home	Home page
signin	Sign in page
signup	Sign up page
listings	Browse all listings
listing-details	View details of a listing
create-listing	Create a new listing (auth required)
edit-listing	Edit an existing listing (auth required)
my-listings	View seller’s own listings (auth required)
sent-offers	View all offers sent (auth required)
sent-offers-details	View details of an offer (auth required)

4.4 Database Models

The system stores data in MongoDB using Mongoose schemas.

Model	Description
user	Stores account credentials and rating statistics.
listing	Represents an item listed for sale.
offer	Represents an offer made by a buyer on a listing.
transaction	Represents a sold listing with an accepted offer.
rating	Represents a rating on a completed transaction with another user.
message	Represents a message made between a buyer or seller on a listing.

5. CRC Cards

Class Name: User	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Register user accounts • Authenticate login • Store username and email • Track user ratings • View profile information	Collaborators: • Listing • Offer • Transaction • Rating • Message

Class Name: Listing	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Store listing information • Track listing available status • Allow seller to edit or delete listing • Display listing details to buyers	Collaborators: • User • Offer • Transaction • Message

Class Name: Offer	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Store offer amount • Track offer status • Allow buyers to cancel offers • Allow sellers to accept or reject offers	Collaborators: • User • Listing • Transaction

Class Name: Transaction	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Record completed sale • Link buyer, seller, listing, and offer to a completed listing • Store listing price, offer price, date • Record ratings provided by seller and lister for the transaction • Prevent duplicate transactions	Collaborators: • User • Listing • Offer

Class Name: Message	
Parent Class (if any): None Subclasses (if any): None	
Responsibilities: • Create and send message between users • Associate message with sender, receiver, and listing • Validate message content is non-empty • Record message with timestamp • Support retrieval of messages grouped by conversation and reply within existing thread • Only involved sender/receiver can view messages	Collaborators: • User • Listing

6. Error Handling Strategy

The LionsList system includes several mechanisms for detecting and responding to errors that may occur during normal operation.

These errors may originate from user input, authentication failures, network communication issues, or database problems. The system is designed so that failures are handled safely and communicated clearly to the user without exposing unnecessary internal system details.

Error handling occurs at both the backend (Express server) and frontend (React client) layers.

6.1 Invalid User Input

Users may enter invalid or incomplete information when registering accounts, creating listings, or submitting offers.

- Username already exists
- Email address not belonging to the York University domain
- Password shorter than the required length
- Missing listing information (title, description, price)

- Invalid offer amount (non-positive values)

The backend implements input validation using Mongoose schema rules and server-side checks. For example, the User schema ensures:

- usernames are unique
- usernames only contain lowercase letters and numbers
- email addresses must end in *@yorku.ca* or *@my.yorku.ca*

When invalid input is detected:

1. The backend returns an HTTP 400 (Bad Request) response.
2. An error message is included in the response body.
3. The frontend displays the message to the user and prevents form submission until the issue is corrected.

This ensures that invalid data is not stored in the database.

6.2 Authentication and Authorization Errors

Certain actions require a user to be authenticated:

- creating a listing
- sending a message
- submitting an offer
- accessing user-specific pages (My Listings, Sent Offers) without authentication

If a user attempts to perform these actions without being logged in, the backend returns an HTTP 401 (Unauthorized) response.

The frontend detects this response and redirects the user to the login page.

Logout functionality removes the authentication token stored in the browser, ensuring the user can no longer access protected actions.

6.3 Database Errors

Database errors may occur if MongoDB becomes unavailable or if database operations fail.

Possible errors include MongoDB connection failure, duplicate username or email conflicts, failed data writes, etc.

When a database error occurs:

1. The backend logs the error to the server console for debugging.
2. The API returns an HTTP 500 (Internal Server Error) response.
3. The frontend displays a generic error message to the user.

Sensitive system information is not exposed to the user for security reasons.

6.4 Network and API Communication Errors

Communication between the frontend and backend occurs through HTTP requests using Axios.

Possible failures include the server being unreachable, network timeouts, incorrect API endpoint, etc.

When such errors occur:

1. Axios detects the request failure.
2. The frontend displays a message indicating that the request failed.
3. The user may retry the operation.

6.5 Data Integrity Protection

To maintain data integrity, the system enforces constraints at multiple levels.

- unique usernames and email addresses in the User schema
- listing ownership verification before editing or deleting a listing
- validation rules for message content and listing information
- ensure only authorized users can modify or delete listings
- ensure only pending offers can be cancelled

These constraints ensure that invalid or unauthorized data cannot corrupt the system state.